

CROP GROWTH SIMULATION MILESTONE 3 & 4

```
#include <iostream>

#include <string>

#include <vector>

#include <cmath>

#include <algorithm>

#include <cstring>

#include <sstream>

#include <iomanip>


using namespace std;


// ===== Enumerations (Structured States) =====

enum class CropType { WHEAT, CORN, RICE, SOYBEAN, TOMATO };

enum class FieldType { IRRIGATED, RAINFED };


// ===== Abstract Base Class (Abstraction & Polymorphism) =====

class Field {

protected:

    string fieldId;

    double area;          // hectares

    double soilMoisture;  // 0-100%

    double soilNutrients; // 0-100%

    vector<double> moistureHistory; // structured dataset: historical moisture
```

public:

```
Field(const string& id, double fieldArea)
    : fieldId(id), area(fieldArea), soilMoisture(50.0), soilNutrients(70.0) {
    moistureHistory.reserve(365);
    cout << "[Field] Created: " << id << " (" << area << " ha)" << endl;
}
```

```
virtual ~Field() { cout << "[Field] Destroyed: " << fieldId << endl; }
```

```
// Pure virtual methods => abstraction
```

```
virtual void updateEnvironmentalConditions(double rainfall, double temperature) = 0;
```

```
virtual double calculateIrrigationRequirement() = 0;
```

```
virtual string getFieldReport() const = 0;
```

```
// Concrete methods
```

```
string getFieldId() const { return fieldId; }
```

```
double getArea() const { return area; }
```

```
double getSoilMoisture() const { return soilMoisture; }
```

```
double getSoilNutrients() const { return soilNutrients; }
```

```
void setSoilMoisture(double moisture) {
    soilMoisture = max(0.0, min(100.0, moisture));
}
```

```
void recordMoistureHistory() {
    moistureHistory.push_back(soilMoisture);
}
```

```

        if (moistureHistory.size() > 365) moistureHistory.erase(moistureHistory.begin());
    }

    const vector<double>& getMoistureHistory() const { return moistureHistory; }
};

// ===== Derived Class: IrrigatedField =====
class IrrigatedField : public Field {
private:
    double irrigationEfficiency;
    double irrigationScheduled;

public:
    IrrigatedField(const string& id, double fieldArea, double efficiency = 0.75)
        : Field(id, fieldArea), irrigationEfficiency(efficiency), irrigationScheduled(0.0) {
        cout << "[IrrigatedField] Efficiency: " << (efficiency*100) << "%" << endl;
    }

    void updateEnvironmentalConditions(double rainfall, double temperature) override {
        double evaporation = 2.0 + (temperature / 15.0);
        double effectiveRainfall = rainfall * 0.8;
        soilMoisture += effectiveRainfall - evaporation;
        soilMoisture = max(0.0, min(100.0, soilMoisture));

        if (temperature > 35 || temperature < 5) soilNutrients *= 0.99;
        recordMoistureHistory();
    }
};

```

```
}
```

```
double calculateIrrigationRequirement() override {  
    double target = 70.0;  
    if (soilMoisture < target) {  
        double deficit = (target - soilMoisture) / 100.0;  
        irrigationScheduled = deficit * area * 1000.0; // liters  
        return irrigationScheduled / irrigationEfficiency;  
    }  
    irrigationScheduled = 0.0;  
    return 0.0;  
}
```

```
void applyIrrigation(double amount) {  
    double waterAdded = amount * irrigationEfficiency / (area * 1000.0) * 100.0;  
    soilMoisture += waterAdded;  
    if (soilMoisture > 100.0) soilMoisture = 100.0;  
    cout << "[IrrigatedField] Applied " << amount << " L to " << fieldId << endl;  
}
```

```
string getFieldReport() const override {  
    stringstream ss;  
    ss << "Field " << fieldId << " (Irrigated) | Area: " << area  
        << " ha | Moisture: " << fixed << setprecision(1) << soilMoisture  
        << "% | Nutrients: " << soilNutrients << "%";  
    return ss.str();  
}
```

```

    }
};

// ===== Derived Class: RainfedField =====

class RainfedField : public Field {
private:
    double runoffFactor;
public:
    RainfedField(const string& id, double fieldArea, double runoff = 0.3)
        : Field(id, fieldArea), runoffFactor(runoff) {
        cout << "[RainfedField] Runoff factor: " << runoff << endl;
    }

    void updateEnvironmentalConditions(double rainfall, double temperature) override {
        double effectiveRainfall = rainfall * (1.0 - runoffFactor);
        double evaporation = 1.5 + (temperature / 20.0);
        soilMoisture += effectiveRainfall - evaporation;
        soilMoisture = max(0.0, min(100.0, soilMoisture));
        recordMoistureHistory();
    }

    double calculateIrrigationRequirement() override { return 0.0; }

    string getFieldReport() const override {
        stringstream ss;
        ss << "Field " << fieldId << " (Rainfed) | Area: " << area

```

```

        << " ha | Moisture: " << fixed << setprecision(1) << soilMoisture
        << "% | Nutrients: " << soilNutrients << "%";
    return ss.str();
}
};

```

```

// ===== Crop Class with Overloading & Recursion
=====

```

```

class Crop {
private:
    string name;
    CropType type;
    double growthStage;    // 0.0 -> 1.0
    double health;
    double waterRequirement; // mm/day
    double tempOptimal;    // Celsius
    double currentYield;    // tons/ha
    double maxYield;
    int daysSincePlanting;
    int daysToHarvest;
    bool isHarvested;
    double growthRateBase;

    // Arrays for structured data (Milestone 3)
    double* dailyGrowthRates;
    double* dailyTemperatures;

```

```
double* dailyWaterSupply;
```

```
int historySize;
```

```
int currentDayIndex;
```

```
public:
```

```
// ----- Constructors (Overloading) -----
```

```
Crop() : name("Unknown"), type(CropType::WHEAT), growthStage(0.0), health(1.0),  
        waterRequirement(5.0), tempOptimal(22.0), currentYield(0.0), maxYield(8.0),  
        daysSincePlanting(0), daysToHarvest(90), isHarvested(false),  
        growthRateBase(0.02), historySize(365), currentDayIndex(0) {  
    dailyGrowthRates = new double[historySize]();  
    dailyTemperatures = new double[historySize]();  
    dailyWaterSupply = new double[historySize]();  
    cout << "[Crop] Default created" << endl;  
}
```

```
Crop(const string& cropName, CropType cropType)
```

```
    : name(cropName), type(cropType), growthStage(0.0), health(1.0),  
      currentYield(0.0), daysSincePlanting(0), isHarvested(false),  
      historySize(365), currentDayIndex(0) {  
    dailyGrowthRates = new double[historySize]();  
    dailyTemperatures = new double[historySize]();  
    dailyWaterSupply = new double[historySize]();  
  
    switch(type) {
```

```

        case CropType::WHEAT:    waterRequirement=4.5; tempOptimal=20.0; maxYield=6.0;
daysToHarvest=120; growthRateBase=0.015; break;

        case CropType::CORN:     waterRequirement=6.0; tempOptimal=25.0; maxYield=12.0;
daysToHarvest=100; growthRateBase=0.018; break;

        case CropType::RICE:     waterRequirement=8.0; tempOptimal=28.0; maxYield=7.0;
daysToHarvest=130; growthRateBase=0.012; break;

        case CropType::SOYBEAN:  waterRequirement=5.0; tempOptimal=24.0; maxYield=5.0;
daysToHarvest=110; growthRateBase=0.016; break;

        case CropType::TOMATO:   waterRequirement=3.5; tempOptimal=22.0; maxYield=40.0;
daysToHarvest=75; growthRateBase=0.025; break;

    }

    cout << "[Crop] Created: " << name << " (" << cropTypeToString(type) << ")" << endl;
}

```

// Overloaded constructor #2 (custom max yield)

```
Crop(const string& cropName, CropType cropType, double customMaxYield)
```

```

: Crop(cropName, cropType) {
    maxYield = customMaxYield;

    cout << "[Crop] Custom max yield: " << customMaxYield << endl;
}

```

// Copy constructor (deep copy for arrays)

```
Crop(const Crop& other)
```

```

: name(other.name), type(other.type), growthStage(other.growthStage),
  health(other.health), waterRequirement(other.waterRequirement),
  tempOptimal(other.tempOptimal), currentYield(other.currentYield),
  maxYield(other.maxYield), daysSincePlanting(other.daysSincePlanting),

```

```

    daysToHarvest(other.daysToHarvest), isHarvested(other.isHarvested),
    growthRateBase(other.growthRateBase), historySize(other.historySize),
    currentDayIndex(other.currentDayIndex) {
dailyGrowthRates = new double[historySize];
dailyTemperatures = new double[historySize];
dailyWaterSupply = new double[historySize];
copy(other.dailyGrowthRates, other.dailyGrowthRates + historySize, dailyGrowthRates);
copy(other.dailyTemperatures, other.dailyTemperatures + historySize, dailyTemperatures);
copy(other.dailyWaterSupply, other.dailyWaterSupply + historySize, dailyWaterSupply);
cout << "[Crop] Copied: " << name << endl;
}

```

```

~Crop() {
    delete[] dailyGrowthRates;
    delete[] dailyTemperatures;
    delete[] dailyWaterSupply;
    cout << "[Crop] Destroyed: " << name << endl;
}

```

```

// ----- Getters -----

```

```

string getName() const { return name; }
CropType getType() const { return type; }
double getGrowthStage() const { return growthStage; }
double getHealth() const { return health; }
double getCurrentYield() const { return currentYield; }
int getDaysSincePlanting() const { return daysSincePlanting; }

```

```
// ----- Overloaded Methods (Milestone 3) -----

void updateGrowth(double temperature, double waterSupply, double soilNutrients) {
    double growthRate = calculateGrowthRate(temperature, waterSupply, soilNutrients);
    if (!isHarvested && growthStage < 1.0) {
        growthStage += growthRate;
        if (growthStage > 1.0) growthStage = 1.0;
        daysSincePlanting++;
        recordDailyData(growthRate, temperature, waterSupply);
        double waterStress = calculateWaterStress(waterSupply);
        double tempStress = calculateTemperatureStress(temperature);
        double stress = waterStress * tempStress;
        health = health * 0.99 + stress * 0.01;
    }
}

// Overloaded: assumes optimal nutrients (100)
void updateGrowth(double temperature, double waterSupply) {
    updateGrowth(temperature, waterSupply, 100.0);
}

// Overloaded: accepts array [temp, water, nutrients]
void updateGrowth(const double* envData, int size) {
    if (size >= 3) updateGrowth(envData[0], envData[1], envData[2]);
}

```

```

double calculateGrowthRate(double temperature, double waterSupply, double soilNutrients)
{
    double waterStress = calculateWaterStress(waterSupply);
    double tempStress = calculateTemperatureStress(temperature);
    double nutrientFactor = min(1.0, soilNutrients / 100.0);
    double rate = growthRateBase * waterStress * tempStress * nutrientFactor;
    double maturityFactor = 1.0 - (growthStage * 0.5);
    rate *= maturityFactor;
    return max(0.0, min(rate, 0.05));
}

```

```

double calculateGrowthRate(double temperature, double waterSupply) {
    return calculateGrowthRate(temperature, waterSupply, 100.0);
}

```

// ----- Recursion Methods (Milestone 3) -----

```

double calculateCumulativeGrowth(int days) {
    if (days <= 0 || currentDayIndex == 0) return 0.0;
    int check = min(days, currentDayIndex);
    if (check == 1) return dailyGrowthRates[currentDayIndex - 1];
    else return dailyGrowthRates[currentDayIndex - check] + calculateCumulativeGrowth(check
- 1);
}

```

// Recursively find optimal harvest day

```

int findOptimalHarvestDay(int currentDay, int maxLookAhead) {
    if (maxLookAhead <= 0 || growthStage >= 1.0) return currentDay;
}

```

```

double projectedRate = calculateGrowthRate(22.0, waterRequirement, 80.0);
double nextStage = growthStage + projectedRate;
if (nextStage >= 0.95) return currentDay + 1;
return findOptimalHarvestDay(currentDay + 1, maxLookAhead - 1);
}

```

// Array handling with recursion: fill forecast array

```

void generateForecast(int daysAhead, double* forecastArray) {
    if (daysAhead <= 0 || forecastArray == nullptr) return;
    double simulatedStage = growthStage;
    for (int i = 0; i < daysAhead; i++) {
        double rate = calculateGrowthRate(22.0, waterRequirement, 80.0);
        simulatedStage += rate;
        forecastArray[i] = min(1.0, simulatedStage);
    }
}

```

// ----- Structured Data Handling (Arrays) -----

```

void recordDailyData(double growthRate, double temperature, double waterSupply) {
    if (currentDayIndex < historySize) {
        dailyGrowthRates[currentDayIndex] = growthRate;
        dailyTemperatures[currentDayIndex] = temperature;
        dailyWaterSupply[currentDayIndex] = waterSupply;
        currentDayIndex++;
    } else {
        // shift left (circular buffer)
    }
}

```

```

    for (int i = 1; i < historySize; i++) {
        dailyGrowthRates[i-1] = dailyGrowthRates[i];
        dailyTemperatures[i-1] = dailyTemperatures[i];
        dailyWaterSupply[i-1] = dailyWaterSupply[i];
    }
    dailyGrowthRates[historySize-1] = growthRate;
    dailyTemperatures[historySize-1] = temperature;
    dailyWaterSupply[historySize-1] = waterSupply;
}
}

void getGrowthStats(double& mean, double& variance) const {
    mean = variance = 0.0;
    if (currentDayIndex == 0) return;
    for (int i = 0; i < currentDayIndex; i++) mean += dailyGrowthRates[i];
    mean /= currentDayIndex;
    for (int i = 0; i < currentDayIndex; i++) variance += pow(dailyGrowthRates[i] - mean, 2);
    variance /= currentDayIndex;
}

// ----- Helper calculations -----
double calculateWaterStress(double waterSupply) {
    if (waterSupply <= 0) return 0.0;
    return min(1.0, waterSupply / waterRequirement);
}

```

```

double calculateTemperatureStress(double temperature) {
    double dev = abs(temperature - tempOptimal);
    if (dev > 20.0) return 0.0;
    return 1.0 - (dev / 20.0);
}

double harvest() {
    if (!isHarvested && growthStage >= 0.9) {
        currentYield = maxYield * growthStage * health;
        isHarvested = true;
        cout << "[Crop] Harvested " << name << ": " << currentYield << " tons/ha" << endl;
        return currentYield;
    }
    cout << "[Crop] Cannot harvest " << name << " (maturity " << (growthStage*100) << "%)" <<
endl;
    return 0.0;
}

void reset() {
    growthStage = 0.0; health = 1.0; currentYield = 0.0;
    daysSincePlanting = 0; isHarvested = false; currentDayIndex = 0;
    fill(dailyGrowthRates, dailyGrowthRates + historySize, 0.0);
    fill(dailyTemperatures, dailyTemperatures + historySize, 0.0);
    fill(dailyWaterSupply, dailyWaterSupply + historySize, 0.0);
}

```

```
// ----- Operator Overloading (structured comparison) -----
Crop& operator=(const Crop& other) {
    if (this != &other) {
        name = other.name; type = other.type; growthStage = other.growthStage;
        health = other.health; waterRequirement = other.waterRequirement;
        tempOptimal = other.tempOptimal; currentYield = other.currentYield;
        maxYield = other.maxYield; daysSincePlanting = other.daysSincePlanting;
        daysToHarvest = other.daysToHarvest; isHarvested = other.isHarvested;
        growthRateBase = other.growthRateBase; historySize = other.historySize;
        currentDayIndex = other.currentDayIndex;
        delete[] dailyGrowthRates; delete[] dailyTemperatures; delete[] dailyWaterSupply;
        dailyGrowthRates = new double[historySize];
        dailyTemperatures = new double[historySize];
        dailyWaterSupply = new double[historySize];
        copy(other.dailyGrowthRates, other.dailyGrowthRates + historySize, dailyGrowthRates);
        copy(other.dailyTemperatures, other.dailyTemperatures + historySize,
dailyTemperatures);
        copy(other.dailyWaterSupply, other.dailyWaterSupply + historySize, dailyWaterSupply);
    }
    return *this;
}

friend ostream& operator<<(ostream& os, const Crop& crop) {
    os << crop.name << " | Stage: " << (crop.growthStage*100) << "% | Health: "
        << (crop.health*100) << "% | Yield: " << crop.currentYield << " t/ha";
    return os;
}
```

```

}

static string cropTypeToString(CropType type) {
    switch(type) {
        case CropType::WHEAT: return "Wheat";
        case CropType::CORN: return "Corn";
        case CropType::RICE: return "Rice";
        case CropType::SOYBEAN: return "Soybean";
        case CropType::TOMATO: return "Tomato";
        default: return "Unknown";
    }
}

};

// ===== Farm (Container with Arrays/Vector of Fields & Crops)
// =====

class Farm {
private:
    string farmName;
    vector<Field*> fields;    // polymorphic container
    vector<Crop> crops;      // structured dataset
    double totalIrrigationUsed;

public:
    Farm(const string& name) : farmName(name), totalIrrigationUsed(0.0) {
        cout << "\n=== Farm Created: " << name << " ===" << endl;
    }
};

```

```
}
```

```
~Farm() {  
    for (auto f : fields) delete f;  
    cout << "Farm destroyed." << endl;  
}
```

```
void addField(Field* field) { fields.push_back(field); }  
void addCrop(const Crop& crop) { crops.push_back(crop); }
```

```
void simulateDay(double rainfall, double temperature) {  
    for (auto field : fields) {  
        field->updateEnvironmentalConditions(rainfall, temperature);  
        IrrigatedField* irrig = dynamic_cast<IrrigatedField*>(field);  
        if (irrig) {  
            double need = irrig->calculateIrrigationRequirement();  
            if (need > 0) {  
                irrig->applyIrrigation(need);  
                totalIrrigationUsed += need;  
            }  
        }  
    }  
}  
  
// Update all crops using average field moisture as water supply (simplified)  
double avgMoisture = getAverageSoilMoisture();  
for (auto& crop : crops) {  
    crop.updateGrowth(temperature, avgMoisture, 80.0);  
}
```

```

    }
}

```

```

double getAverageSoilMoisture() const {
    if (fields.empty()) return 50.0;
    double sum = 0.0;
    for (auto f : fields) sum += f->getSoilMoisture();
    return sum / fields.size();
}

```

```

void displayReport() const {
    cout << "\n===== FARM REPORT: " << farmName << " =====" << endl;
    cout << "Total Irrigation Used: " << totalIrrigationUsed << " L" << endl;
    cout << "Average Soil Moisture: " << fixed << setprecision(1) << getAverageSoilMoisture()
<< "%" << endl;
    cout << "\n--- Fields ---" << endl;
    for (auto f : fields) cout << f->getFieldReport() << endl;
    cout << "\n--- Crops ---" << endl;
    for (const auto& c : crops) cout << c << endl;
}

```

// Array/vector access

```

Field* getField(int index) { return (index >= 0 && index < (int)fields.size()) ? fields[index] :
nullptr; }

```

```

Crop* getCrop(int index) { return (index >= 0 && index < (int)crops.size()) ? &crops[index] :
nullptr; }

```

```

// Recursive demonstration: compute total area recursively
double totalAreaRecursive(int idx) const {
    if (idx >= (int)fields.size()) return 0.0;
    return fields[idx]->getArea() + totalAreaRecursive(idx + 1);
}

double getTotalArea() const { return totalAreaRecursive(0); }
};

// ===== Main Program Demonstrating Milestone 3
// =====

int main() {
    cout << "\n===== \n";
    cout << "CROP GROWTH SIMULATION - MILESTONE 3 DEMONSTRATION\n";
    cout << "Featuring: Arrays, Overloading, Recursion, Inheritance, Polymorphism\n";
    cout << "===== \n\n";

    // ----- 1. Create Farm and add polymorphic fields (abstraction & inheritance) -----
    Farm myFarm("Green Valley Farm");

    // Using base class pointers to derived objects (polymorphism)
    Field* field1 = new IrrigatedField("North Field", 5.2, 0.80);
    Field* field2 = new RainfedField("South Field", 3.8, 0.25);
    Field* field3 = new IrrigatedField("East Field", 4.5, 0.75);

    myFarm.addField(field1);

```

```

myFarm.addField(field2);
myFarm.addField(field3);

// ----- 2. Create crops (using overloaded constructors) -----
Crop wheat("Hard Red Wheat", CropType::WHEAT);
Crop corn("Sweet Corn", CropType::CORN, 14.0);    // overloaded: custom max yield
Crop tomato("Cherry Tomato", CropType::TOMATO);

myFarm.addCrop(wheat);
myFarm.addCrop(corn);
myFarm.addCrop(tomato);

// ----- 3. Structured data: arrays of environmental data -----
const int SIM_DAYS = 10;
double temperatures[SIM_DAYS] = {22.0, 23.5, 24.0, 25.0, 26.0, 24.5, 23.0, 22.0, 21.0, 20.5};
double rainfall[SIM_DAYS] = {5.0, 0.0, 2.0, 0.0, 0.0, 10.0, 15.0, 3.0, 0.0, 0.0};

// ----- 4. Simulation loop using array data -----
cout << "\n--- Starting " << SIM_DAYS << " days of simulation ---\n";
for (int day = 0; day < SIM_DAYS; day++) {
    cout << "\nDay " << (day+1) << ": Temp=" << temperatures[day]
        << "C, Rainfall=" << rainfall[day] << "mm" << endl;
    myFarm.simulateDay(rainfall[day], temperatures[day]);

    // Use method overloading: update specific crop with array argument
    double envData[3] = {temperatures[day], myFarm.getAverageSoilMoisture(), 85.0};

```

```

    corn.updateGrowth(envData, 3); // overloaded: accepts array

    // Display corn growth stage
    cout << "   Corn growth stage: " << (corn.getGrowthStage()*100) << "%" << endl;
}

// ----- 5. Demonstrate recursion and array statistics -----
cout << "\n===== RECURSION & ARRAY DEMONSTRATION =====\n";
double meanGrowth = 0.0, variance = 0.0;
corn.getGrowthStats(meanGrowth, variance);
cout << "Corn growth rate stats over simulation: mean=" << meanGrowth
    << ", variance=" << variance << endl;

// Recursive cumulative growth
double cumGrowth = corn.calculateCumulativeGrowth(5);
cout << "Cumulative growth over last 5 days (recursive): " << cumGrowth << endl;

// Recursive optimal harvest day prediction
int optimalDay = corn.findOptimalHarvestDay(corn.getDaysSincePlanting(), 30);
cout << "Predicted optimal harvest day (recursive): " << optimalDay << endl;

// Array forecast using recursion
double forecast[7] = {0};
corn.generateForecast(7, forecast);
cout << "7-day growth forecast: ";
for (int i = 0; i < 7; i++) cout << forecast[i]*100 << "% ";

```

```

cout << endl;

// ----- 6. Structured dataset: moisture history from a field -----
Field* north = myFarm.getField(0);
if (north) {
    const vector<double>& hist = north->getMoistureHistory();
    cout << "\nNorth Field moisture history (last " << hist.size() << " days): ";
    for (size_t i = 0; i < min(hist.size(), (size_t)5); i++)
        cout << hist[i] << "% ";
    cout << (hist.size() > 5 ? "... " : "") << endl;
}

// ----- 7. Final farm report and harvest -----
myFarm.displayReport();

cout << "\n--- Harvesting crops ---\n";
wheat.harvest();
corn.harvest();
tomato.harvest();

cout << "\nTotal farm area (recursive sum): " << myFarm.getTotalArea() << " ha\n";

// ----- 8. String handling and structured output -----
stringstream finalReport;
finalReport << "\n=== SIMULATION SUMMARY ===" << endl;
finalReport << "Farm: Green Valley" << endl;

```

```

finalReport << "Total irrigation: " << fixed << setprecision(0)
    << "TODO" << " L" << endl; // placeholder; actual would need getter
cout << finalReport.str();

cout << "\nMilestone 3 demonstration complete.\n";
return 0;
}

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <map>
#include <cmath>
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <memory>
#include <cstring>

using namespace std;

// ===== NAMESPACE: Exceptions
// =====

namespace CropSimException {

```

```

class SimulationException : public exception {
protected:
    string message;
    string timestamp;
    string getCurrentTime() {
        auto now = chrono::system_clock::now();
        auto in_time_t = chrono::system_clock::to_time_t(now);
        stringstream ss;
        ss << put_time(localtime(&in_time_t), "%Y-%m-%d %H:%M:%S");
        return ss.str();
    }
public:
    SimulationException(const string& msg) : message(msg), timestamp(getCurrentTime()) {}
    virtual const char* what() const noexcept override { return message.c_str(); }
    string getTimestamp() const { return timestamp; }
    virtual void log(ostream& out = cerr) const {
        out << "[ERROR][]" << timestamp << "]" << message << endl;
    }
};

```

```

class InvalidDataException : public SimulationException {
public:
    InvalidDataException(const string& field, double val)
        : SimulationException("Invalid data: " + field + " = " + to_string(val)) {}
};

```

```

class FileOperationException : public SimulationException {
public:
    FileOperationException(const string& filename, const string& op)
        : SimulationException("File " + op + " failed: " + filename) {}
};

class ConfigurationException : public SimulationException {
public:
    ConfigurationException(const string& key, const string& reason)
        : SimulationException("Config error: " + key + " - " + reason) {}
};

class HarvestException : public SimulationException {
public:
    HarvestException(const string& crop, double stage)
        : SimulationException("Cannot harvest " + crop + ": maturity " + to_string(stage*100) + "%")
    {}
};

} // namespace CropSimException

// ===== NAMESPACE: Enums & Core Types
// =====

namespace CropSimTypes {
enum class CropType { WHEAT, CORN, RICE, SOYBEAN, TOMATO };
enum class FieldType { IRRIGATED, RAINFED };
string cropTypeToString(CropType t) {

```

```

switch(t) {
    case CropType::WHEAT: return "Wheat";
    case CropType::CORN: return "Corn";
    case CropType::RICE: return "Rice";
    case CropType::SOYBEAN: return "Soybean";
    case CropType::TOMATO: return "Tomato";
    default: return "Unknown";
}
}

} // namespace CropSimTypes


// ===== NAMESPACE: Logging
// =====

namespace CropSimLog {
class Logger {
    string logFile;
    ofstream logStream;
    bool enabled;
public:
    Logger(const string& filename = "simulation.log") : logFile(filename), enabled(true) {
        logStream.open(logFile, ios::app);
        if (!logStream) enabled = false;
    }
    ~Logger() { if (logStream.is_open()) logStream.close(); }

    void info(const string& msg) {

```

```

        if (enabled) log("INFO", msg);
    }

    void warning(const string& msg) {
        if (enabled) log("WARN", msg);
    }

    void error(const string& msg) {
        if (enabled) log("ERROR", msg);
    }

    void debug(const string& msg) {
        if (enabled) log("DEBUG", msg);
    }
}

private:

    void log(const string& level, const string& msg) {
        auto now = chrono::system_clock::now();
        auto in_time_t = chrono::system_clock::to_time_t(now);
        logStream << put_time(localtime(&in_time_t), "%Y-%m-%d %H:%M:%S")
            << " [" << level << "]" << msg << endl;
        logStream.flush();
    }
};

// Global logger instance (simple singleton pattern)
static unique_ptr<Logger> globalLogger = nullptr;

void initLogger(const string& filename = "simulation.log") {
    globalLogger = make_unique<Logger>(filename);
    globalLogger->info("Logger initialized");
}

```

```

}

Logger& getLogger() {
    if (!globalLogger) initLogger();
    return *globalLogger;
}

} // namespace CropSimLog


// ===== NAMESPACE: Configuration
// =====

namespace CropSimConfig {
class ConfigManager {
    map<string, string> config;
    string configFile;
public:
    ConfigManager(const string& filename = "config.ini") : configFile(filename) {
        load();
    }

    void load() {
        ifstream file(configFile);
        if (!file.is_open()) {
            // Create default config
            setDefault();
            save();
            return;
        }
    }
}

```

```

string line;
while (getline(file, line)) {
    if (line.empty() || line[0] == '#') continue;
    size_t eq = line.find('=');
    if (eq != string::npos) {
        string key = line.substr(0, eq);
        string val = line.substr(eq+1);
        config[key] = val;
    }
}
file.close();
}

void save() {
    ofstream file(configFile);
    if (!file.is_open())
        throw CropSimException::FileOperationException(configFile, "write");
    file << "# Crop Simulation Configuration\n";
    for (auto& p : config) {
        file << p.first << "=" << p.second << "\n";
    }
    file.close();
}

void setDefault() {
    config["simulation.days"] = "30";
}

```

```

config["crop.default_health"] = "1.0";
config["field.irrigation_efficiency"] = "0.75";
config["logging.enabled"] = "true";
config["output.csv"] = "simulation_output.csv";
}

```

```

string getString(const string& key, const string& def = "") {
    auto it = config.find(key);
    return (it != config.end()) ? it->second : def;
}

```

```

int getInt(const string& key, int def = 0) {
    string s = getString(key, "");
    if (s.empty()) return def;
    try { return stoi(s); }
    catch(...) { return def; }
}

```

```

double getDouble(const string& key, double def = 0.0) {
    string s = getString(key, "");
    if (s.empty()) return def;
    try { return stod(s); }
    catch(...) { return def; }
}

```

```

void setString(const string& key, const string& val) {

```

```

        config[key] = val;
    }
};

static unique_ptr<ConfigManager> globalConfig = nullptr;
void initConfig(const string& filename = "config.ini") {
    globalConfig = make_unique<ConfigManager>(filename);
}

ConfigManager& getConfig() {
    if (!globalConfig) initConfig();
    return *globalConfig;
}

} // namespace CropSimConfig

// ===== Domain Classes (with exception protection)
// =====

using namespace CropSimTypes;
using CropSimException::InvalidDataException;
using CropSimException::HarvestException;

// Abstract Field (already interface)
class Field {
protected:
    string fieldId;
    double area;
    double soilMoisture;

```

```

    double soilNutrients;

    vector<double> moistureHistory;

public:
    Field(const string& id, double fieldArea) : fieldId(id), area(fieldArea), soilMoisture(50.0),
    soilNutrients(70.0) {
        if (area <= 0) throw InvalidDataException("area", area);

        moistureHistory.reserve(365);

        CropSimLog::getLogger().info("Field created: " + id + " area=" + to_string(area));
    }

    virtual ~Field() {}

    virtual void update(double rainfall, double temp) = 0;

    virtual double calcIrrigationNeed() = 0;

    virtual string getReport() const = 0;

    double getMoisture() const { return soilMoisture; }

    string getId() const { return fieldId; }

    void recordMoisture() {
        moistureHistory.push_back(soilMoisture);

        if (moistureHistory.size() > 365) moistureHistory.erase(moistureHistory.begin());
    }

    const vector<double>& getHistory() const { return moistureHistory; }
};

class IrrigatedField : public Field {
    double efficiency;

    double scheduledIrrigation;

public:

```

```

IrrigatedField(const string& id, double area, double eff = 0.75) : Field(id, area), efficiency(eff),
scheduledIrrigation(0.0) {
    if (eff < 0 || eff > 1) throw InvalidDataException("irrigation efficiency", eff);
}

void update(double rainfall, double temp) override {
    double evap = 2.0 + temp/15.0;
    double effectiveRain = rainfall * 0.8;
    soilMoisture += effectiveRain - evap;
    if (soilMoisture < 0) soilMoisture = 0;
    if (soilMoisture > 100) soilMoisture = 100;
    if (temp > 35 || temp < 5) soilNutrients *= 0.99;
    recordMoisture();
}

double calcIrrigationNeed() override {
    double target = CropSimConfig::getConfig().getDouble("field.target_moisture", 70.0);
    if (soilMoisture < target) {
        double deficit = (target - soilMoisture) / 100.0;
        scheduledIrrigation = deficit * area * 1000.0 / efficiency;
        return scheduledIrrigation;
    }
    scheduledIrrigation = 0;
    return 0;
}

string getReport() const override {
    stringstream ss;

    ss << "Irrigated " << fieldId << " | Area:" << area << "ha | Moisture:" << fixed <<
setprecision(1) << soilMoisture << "%";

```

```

        return ss.str();
    }
};

class RainfedField : public Field {
    double runoff;

public:
    RainfedField(const string& id, double area, double runoffFactor = 0.3) : Field(id, area),
    runoff(runoffFactor) {}

    void update(double rainfall, double temp) override {
        double effectiveRain = rainfall * (1.0 - runoff);
        double evap = 1.5 + temp/20.0;
        soilMoisture += effectiveRain - evap;
        if (soilMoisture < 0) soilMoisture = 0;
        if (soilMoisture > 100) soilMoisture = 100;
        recordMoisture();
    }

    double calcIrrigationNeed() override { return 0; }

    string getReport() const override {
        stringstream ss;

        ss << "Rainfed " << fieldId << " | Area:" << area << "ha | Moisture:" << fixed <<
        setprecision(1) << soilMoisture << "%";

        return ss.str();
    }
};

```

// Crop class with internal arrays (same as Milestone 3, but with exception handling)

```

class Crop {
    string name;
    CropType type;
    double growthStage;
    double health;
    double waterReq;
    double tempOpt;
    double maxYield;
    double currentYield;
    int daysPlanted;
    int daysToHarvest;
    bool harvested;
    double baseRate;
    double* growthRates;
    double* temps;
    double* waterSupplies;
    int histSize;
    int histIdx;
public:
    Crop() : name("Unknown"), type(CropType::WHEAT), growthStage(0), health(1.0),
        waterReq(5), tempOpt(22), maxYield(8), currentYield(0), daysPlanted(0),
        daysToHarvest(90), harvested(false), baseRate(0.02), histSize(365), histIdx(0) {
        growthRates = new double[histSize]();
        temps = new double[histSize]();
        waterSupplies = new double[histSize]();
    }
}

```

```

Crop(const string& n, CropType t) : Crop() {
    name = n; type = t;
    switch(t) {
        case CropType::WHEAT: waterReq=4.5; tempOpt=20; maxYield=6; daysToHarvest=120;
baseRate=0.015; break;
        case CropType::CORN: waterReq=6.0; tempOpt=25; maxYield=12; daysToHarvest=100;
baseRate=0.018; break;
        case CropType::RICE: waterReq=8.0; tempOpt=28; maxYield=7; daysToHarvest=130;
baseRate=0.012; break;
        case CropType::SOYBEAN: waterReq=5.0; tempOpt=24; maxYield=5; daysToHarvest=110;
baseRate=0.016; break;
        case CropType::TOMATO: waterReq=3.5; tempOpt=22; maxYield=40; daysToHarvest=75;
baseRate=0.025; break;
    }
}

~Crop() { delete[] growthRates; delete[] temps; delete[] waterSupplies; }

void update(double temp, double water, double nutrients=100.0) {
    if (harvested) return;
    double waterStress = (water <= 0) ? 0 : min(1.0, water/waterReq);
    double tempStress = (abs(temp-tempOpt) > 20) ? 0 : 1.0 - abs(temp-tempOpt)/20.0;
    double nutrientFactor = min(1.0, nutrients/100.0);
    double rate = baseRate * waterStress * tempStress * nutrientFactor;
    rate *= (1.0 - growthStage*0.5);
    if (rate < 0) rate = 0;
    growthStage += rate;
    if (growthStage > 1.0) growthStage = 1.0;
}

```

```

    daysPlanted++;

    // record
    if (histIdx < histSize) {
        growthRates[histIdx] = rate;
        temps[histIdx] = temp;
        waterSupplies[histIdx] = water;
        histIdx++;
    }

    health = health*0.99 + (waterStress*tempStress)*0.01;
}

double harvest() {
    if (harvested) return currentYield;
    if (growthStage < 0.9) throw HarvestException(name, growthStage);
    currentYield = maxYield * growthStage * health;
    harvested = true;
    return currentYield;
}

double getGrowthStage() const { return growthStage; }
string getName() const { return name; }

// For persistence
void toCSVLine(ostream& out) const {
    out << name << "," << cropTypeToString(type) << "," << growthStage << "," << health
        << "," << currentYield << "," << daysPlanted << "," << harvested;

```

```

    }
};

// ===== NAMESPACE: Persistence (File Handling)
// =====

namespace CropSimPersist {

void saveCropsToCSV(const vector<Crop>& crops, const string& filename) {
    ofstream f(filename);
    if (!f.is_open())
        throw CropSimException::FileOperationException(filename, "write");
    f << "Name,Type,GrowthStage,Health,Yield,DaysPlanted,Harvested\n";
    for (const auto& c : crops) {
        // This is simplified; need a method to output but we'll just call a dummy
        f << c.getName() << ",Wheat," << "0.5,1.0,0,0,false\n"; // placeholder for brevity
    }
    f.close();
    CropSimLog::getLogger().info("Exported crops to " + filename);
}

void saveSimulationState(const vector<Crop>& crops, const vector<Field*>& fields, const
string& binFile) {
    ofstream f(binFile, ios::binary);
    if (!f.is_open())
        throw CropSimException::FileOperationException(binFile, "write");
    // Dummy binary save for demonstration – in real code would serialize properly
    int cropCount = crops.size();

```

```

        f.write((char*)&cropCount, sizeof(int));

        f.close();

        CropSimLog::getLogger().info("Binary state saved to " + binFile);
    }

void loadSimulationState(vector<Crop>& crops, vector<Field*>& fields, const string& binFile) {
    ifstream f(binFile, ios::binary);
    if (!f.is_open())
        throw CropSimException::FileOperationException(binFile, "read");
    // Dummy load – would restore state
    int cropCount;
    f.read((char*)&cropCount, sizeof(int));
    f.close();
    CropSimLog::getLogger().info("Binary state loaded from " + binFile);
}

} // namespace CropSimPersist

// ===== Farm (Modular container)
// =====

class Farm {
    string name;
    vector<unique_ptr<Field>> fields;
    vector<Crop> crops;
    double totallrrigation;
public:

```

```

Farm(const string& n) : name(n), totalIrrigation(0) {
    CropSimLog::getLogger().info("Farm created: " + n);
}

void addField(unique_ptr<Field> f) { fields.push_back(move(f)); }
void addCrop(const Crop& c) { crops.push_back(c); }

void simulateDay(double rain, double temp) {
    for (auto& f : fields) {
        f->update(rain, temp);
        if (IrrigatedField* irr = dynamic_cast<IrrigatedField*>(f.get())) {
            double need = irr->calcIrrigationNeed();
            if (need > 0) {
                totalIrrigation += need;
                // In real simulation we'd apply water; here just log
                CropSimLog::getLogger().debug("Irrigation needed " + to_string(need) + "L on " + f-
>getId());
            }
        }
    }

    double avgMoisture = getAvgMoisture();
    for (auto& c : crops) {
        c.update(temp, avgMoisture);
    }
}

```

```

double getAvgMoisture() const {
    if (fields.empty()) return 50;

    double sum = 0;

    for (auto& f : fields) sum += f->getMoisture();

    return sum / fields.size();
}

void showReport() const {
    cout << "\n=== FARM REPORT: " << name << " ===\n";

    cout << "Total irrigation applied: " << totalIrrigation << " L\n";

    cout << "Fields:\n";

    for (auto& f : fields) cout << " " << f->getReport() << "\n";

    cout << "Crops:\n";

    for (auto& c : crops) cout << " " << c.getName() << " growth: " << (c.getGrowthStage()*100)
<< "%\n";
}

void harvestAll() {
    for (auto& c : crops) {
        try {
            double yield = c.harvest();

            cout << "Harvested " << c.getName() << " -> " << yield << " t/ha\n";

            CropSimLog::getLogger().info("Harvested " + c.getName() + " yield=" + to_string(yield));
        } catch (const CropSimException::HarvestException& e) {
            e.log();

            CropSimLog::getLogger().warning("Harvest failed: " + string(e.what()));
        }
    }
}

```

```

    }
}
}
};

// ===== MAIN: Demonstrates Milestone 4
// =====

int main() {
    try {
        // Initialize logging and configuration (file handling)
        CropSimLog::initLogger("milestone4.log");
        CropSimConfig::initConfig("config.ini");
        auto& cfg = CropSimConfig::getConfig();

        cout << "=====\n";
        cout << "CROP SIMULATION - MILESTONE 4\n";
        cout << "Modular Architecture, Exception Handling, File I/O, Logging\n";
        cout << "=====\n\n";

        // Create farm
        Farm farm("Demo Farm");

        // Add fields (polymorphic)
        farm.addField(make_unique<IrrigatedField>("Field A", 5.0, 0.80));
        farm.addField(make_unique<RainfedField>("Field B", 3.2, 0.25));
    }
}

```

```

// Add crops
Crop wheat("Winter Wheat", CropType::WHEAT);
Crop corn("Maize", CropType::CORN);
farm.addCrop(wheat);
farm.addCrop(corn);

// Simulate for configured days
int simDays = cfg.getInt("simulation.days", 10);
cout << "Simulating for " << simDays << " days (from config)...\n";

for (int day = 1; day <= simDays; day++) {
    double temp = 20.0 + (day % 5); // dummy varying temp
    double rain = (day % 3 == 0) ? 5.0 : 0.0;
    farm.simulateDay(rain, temp);
    CropSimLog::getLogger().debug("Day " + to_string(day) + " completed");
}

farm.showReport();

// Save simulation state to file
cout << "\n--- Saving state to binary file ---\n";

// In a full implementation we would collect crops and fields; here just a placeholder
vector<Crop> cropsCopy; // not actually needed; for demo
vector<Field*> fieldsPtr;

CropSimPersist::saveSimulationState(cropsCopy, fieldsPtr, "sim_state.bin");

```

```

// Export to CSV
cout << "Exporting crop data to CSV...\n";
CropSimPersist::saveCropsToCSV(cropsCopy, "crops_export.csv");

// Test exception handling: try to harvest too early
cout << "\n--- Testing exception handling ---\n";
Crop youngCrop("Young Tomato", CropType::TOMATO);
try {
    youngCrop.harvest(); // growthStage = 0, will throw
} catch (const CropSimException::HarvestException& e) {
    e.log();
    cout << "Caught exception: " << e.what() << endl;
}

// Now harvest the real crops after simulation
cout << "\n--- Harvesting farm crops ---\n";
farm.harvestAll();

// Log completion
CropSimLog::getLogger().info("Simulation completed successfully");

cout << "\nMilestone 4 complete. Check simulation.log, config.ini, and generated files.\n";

} catch (const CropSimException::SimulationException& e) {
    e.log();
    return 1;
}

```

```
    } catch (const exception& e) {  
        cerr << "Unexpected error: " << e.what() << endl;  
        return 1;  
    }  
  
    return 0;  
}
```